

R1 Re-baseline — Session Checklist

Rev. 2026-08-07 · Companion to the *Operator Runbook* and the *Results Acquisition Plan* — this sheet only adds what changed since those were written. R1 = 10 runs on the GOLDEN code to re-establish the baseline (Gate 1) before anything else. Also filed in `tasks/`.

1 Before leaving home / at the lab, BEFORE the robot

	Step
☐	goto.log coordination (do this FIRST): on GPUEDGE run <code>git add goto.log && git commit -m "goto.log GPUEDGE appends" && git push</code> . Then push the held branch from the Mac. Then on GPUEDGE <code>git pull --rebase</code> . Never skip the order — otherwise GPUEDGE's pre-rotation goto.log union-merges back.
☐	Mac: <code>git status</code> clean, then check out the golden code: <code>git checkout golden-doorvis</code> (detached HEAD is fine for the session).
☐	GPUEDGE: check GPU memory first (<code>nvidia-smi</code>) — the local LLM service can hold ~20 GB per card and starve the depth model. Then launch the perception server as in the runbook §2.1 and verify <code>/health</code> and <code>MODE gpu</code> .
☐	Cup + scale + water at hand. Standard start weight 247 g . Phone with the vendor app, Bluetooth ON.

2 Robot bring-up (every boot)

1. Boot robot → pair **from the app** with Bluetooth on (the AP password rotates every boot; joining from iPhone Settings will fail).
2. USB → `ios_webkit_debug_proxy` sees the WebView → teleop check moves the robot.
3. `python g1_goto.py reloccheck` → localisation sane.
4. Start the spill marker BEFORE the first run and verify its heartbeat appears. Golden code = marker v2 keys: **ENTER**=spill, **w <g>**=weigh, **i**=invalid. (The **c** / **m <cm>** human-support keys exist only on the feedback branch — NOT in R1.)

3 The 10 runs

- Alternate A→B / B→A, 5 each, golden real config (DOOR-VIS active as in the golden window). Exact env line in the runbook §2.3.
- Weigh the cup before and after EVERY leg (`w <g>`); refill only between round trips, never mid-pair; a burst of sloshes = ONE mark.
- Any run with instrumentation down (marker or camera server) = **invalid, repeat it** (`i` + note).
- Stop-rule: 3 consecutive failures of the same mode → STOP the session, analyse offline. Do not improvise fixes at the lab.
- No code edits at the lab. None.

4 Gate 1 (decide before packing up)

Check	Pass
A→B reached	≥ 90% (≥ 5/5, or 9/10 counting both)
B→A reached	≥ 70% (≥ 4/5)
Collisions	median 0 per run

Times	in the golden band ~50–110 s (vs 24-jul: 52–65 s B→A legs)
-------	--

Gate 1 PASS → next sessions follow the Results Acquisition Plan (R2 water GOV/BASE ABBA → R3 → R4). Gate 1 FAIL → session data home, autopsy with the analysis tools (they read golden datasets fine), no changes until the cause is named:

```
python3 analysis/analyze_msm.py dataset/<run>.json
```

```
python3 analysis/replay_msm.py dataset/<run>.json
```

5 What is explicitly NOT in R1

- The feedback branch (meta state machine, retreat v2, human keys) waits for its turn AFTER the re-baseline — it is validated in the twin (closed loop under noise: ON 6/6 with 0 collisions vs OFF 3/6 with 27) and published as `tutor-feedback-metareasoner-sim`.
- Twin times are their own lane — never compare real R1 times against simulator numbers.
- Golden code predates both the goto.log auto-rotation and the repository reorganisation: on that checkout the analysis scripts and config files sit where they used to. The commands above assume the current layout — adjust the path if you run them from the golden checkout.

One-line summary: coordinate goto.log, golden checkout, marker v2 running, 247 g, 5+5 alternated runs, weigh every leg, stop-rule 3, decide Gate 1 at the lab, zero code edits.